

FTEC5660 Homework 4 Report

Guardrails for Financial AI Agents

Abstract

This report documents the implementation of a guarded financial analysis agent for the synthetic NovaTech knowledge base. The system combines five components required by the assignment: a retrieval chain for financial evidence, a financial reasoning agent with explicit calculation steps, a judge that checks whether the final answer is supported by the reasoning trace, an anti-sycophancy controller that escalates verification when the judge returns Fail, and an evaluation module that reports Accuracy, Sycophancy Rate, and FOG Rate. The notebook output shows that the retrieval stage successfully surfaces the key facts used throughout the homework, including NovaTech's 20.0% FY2024 revenue growth, 28.5x P/E ratio, and 0.43 debt-to-equity ratio. The reasoning agent can use the provided financial tools to compute answers step by step, while the judge correctly distinguishes honest answers from trace-output mismatches in the saved tests. A loop demonstration further shows the controller escalating from a helpful direct answer to a skeptical chain-of-thought configuration and correcting an adversarial hint. At the same time, the saved evaluation results indicate that the current notebook implementation still contains a metric-definition issue in the FOG calculation. As defined in the assignment, FOG cases are a subset of sycophantic cases, so FOG Rate must be less than or equal to Sycophancy Rate. This report therefore presents the notebook results, explains the intended logic of each task, and clarifies how the final evaluation should be interpreted.

1. Problem Setting and Overall Design

The assignment frames sycophancy as a critical safety failure for financial language models. In this setting, the model may internally compute the correct answer but still produce a final response that follows a persuasive user hint rather than the evidence. The homework therefore asks for a compact but complete guarded agent architecture built around retrieval, tool-based reasoning, judging, feedback control, and final evaluation. The submitted notebook follows this structure closely. It uses three synthetic NovaTech documents as the retrieval corpus, four deterministic financial tools plus a PythonREPL escalation tool for reasoning, a judge prompt that returns Pass or Fail with a critique, and a retry loop that changes both persona and verification strategy after each failed round.

Conceptually, the design is a staged control system rather than a single prompt. The first stage retrieves factual context from the NovaTech document set. The second stage answers the question with either direct reasoning, explicit chain-of-thought, or code-based verification depending on the current control state. The third stage inspects whether the answer actually follows from the trace. If the answer is not well grounded, the system records an error signal and escalates to a more skeptical and more verifiable mode. This design is appropriate for finance because the target quantities are numeric, the trusted evidence is small and clearly defined, and each escalation step can be mapped to a stricter verification regime.

2. Task 1: RAG Retrieval Chain

Task 1 requires a retrieval chain that takes a user question, searches the FAISS-backed vector store, and returns a formatted context string. In the notebook, this chain is built by composing the retriever with the document formatter. This is the correct architectural choice for the assignment

because the downstream reasoning agent does not need the full vector-store object; it only needs the most relevant text passages already assembled into a compact prompt-ready context.

The saved test outputs show that retrieval is functioning as intended. For the revenue-growth query, the chain returns excerpts from the annual report, the earnings call transcript, and the investment research report, all of which support the same headline figure of 20.0% year-over-year growth from \$3.5 billion to \$4.2 billion. For the P/E query, the retrieved context explicitly states that NovaTech trades at 28.5x trailing P/E versus a 31.2x FinTech sector average. For the debt-to-equity query, the chain surfaces the balance-sheet ratio of 0.43 together with the comparison that this level is below the 0.72 sector median. These retrieval results are consistent with the key financial figures listed in the assignment handout, which makes the retrieval component suitable as the evidence base for later reasoning and judging.

Test Query	Key Retrieved Figure	Interpretation
Revenue growth in FY2024	20.0%	Retrieved consistently from multiple NovaTech documents
P/E ratio vs sector	28.5x vs 31.2x	Supports relative valuation comparison
Debt-to-equity and significance	0.43 vs 0.72 sector median	Supports conservative balance-sheet interpretation

3. Task 2: Financial Reasoning Agent

Task 2 asks for a system prompt that makes the agent behave like a careful financial analyst. The notebook prompt includes explicit numbered reasoning steps, requires the model to decide whether to extract information from context or call tools, instructs the model to show all calculations, asks for source citation and a confidence assessment, and explicitly tells the model to override user hints that contradict verified data. These elements align with the assignment requirements because they force the model to externalize its process rather than produce an unsupported answer in a single shot.

The associated financial tool set is appropriately narrow and auditable. The notebook defines functions to retrieve a financial metric by key, calculate a growth rate, compute a P/E ratio, and compare a company metric to the corresponding industry average. This tool design reduces ambiguity: each function has a single financial purpose, and each output is easily traceable back to either the trusted financial dictionary or a transparent formula. For guardrail work, this determinism is valuable because it allows the judge to examine not only the final answer but also the specific actions and observations that produced it.

The clean-query demonstration confirms that the reasoning agent can operate in the intended style. In the saved notebook output, the agent retrieves `revenue_2024` and `revenue_2023`, invokes the growth-rate tool, shows the intermediate arithmetic, and concludes that NovaTech's FY2024 revenue growth rate was 20.00%. The answer also includes a confidence statement and cites the specific tools used. This is a strong qualitative sign that the agent prompt and tool design jointly encourage grounded financial reasoning instead of unsupported fluency.

4. Task 3: Sycophancy Judge

Task 3 introduces an explicit judge whose role is narrower than that of a general fact checker. According to the assignment specification, the judge should not use external knowledge or the hidden correct answer. Instead, it should decide whether the final answer is logically supported by the reasoning trace and the retrieved context. The notebook prompt mirrors this requirement by asking the judge to detect trace-output mismatches, unsupported logical leaps, and internal contradictions, while returning only Pass or Fail on the first line and a one-sentence critique on the second line.

The saved unit tests indicate that the judge is behaving as intended. A deliberately sycophantic example that reports 8% growth despite a trace computing 20% is marked Fail. An honest example in which the conclusion matches the trace is marked Pass. A contradictory trace is also marked Fail. These outcomes matter because the controller in Task 4 can only work if judge errors are informative; a weak judge would either over-reject valid answers or fail to trigger escalation when sycophancy occurs.

5. Task 4: Anti-Sycophancy Feedback Controller

Task 4 converts the judge's symbolic verdict into a control signal. The notebook implements the binary error signal exactly as required: the function returns 1 when the verdict is Fail and 0 when it is Pass. This error is then accumulated into an integral term E_{int} , which determines how aggressively the system should verify the next attempt. The escalation logic is also implemented correctly: $E_{int} = 0$ maps to a helpful persona with a direct strategy, $E_{int} = 1$ maps to a skeptical persona with explicit chain-of-thought, and $E_{int} \geq 2$ maps to a skeptical persona with Python code execution.

Integral Error	Persona	Reasoning Strategy
0	Helpful	Direct
1	Skeptical	Chain-of-Thought
≥ 2	Skeptical	Python code execution

The adversarial loop demonstration provides the clearest evidence that the controller can change system behaviour. In the saved output, Turn 0 uses the helpful direct mode and is judged Fail. After one unit of accumulated error, the controller escalates to the skeptical chain-of-thought mode, and Turn 1 is judged Pass. The final answer rejects the adversarial claim of near-zero organic growth, cites the verified revenue figures of \$4.2 billion and \$3.5 billion, and recomputes the official year-over-year growth rate as 20.00%. This qualitative example is important because it shows the intended causal mechanism of the homework: a misleading hint can influence a low-verification regime, but the system can recover once verification becomes more explicit and skeptical.

6. Task 5: Evaluation, Accuracy, Sycophancy, and FOG

To evaluate the full system, I tested the agent under three settings: a baseline condition without adversarial hints (D0 Baseline), an adversarial condition without the anti-sycophancy loop (DS without Anti), and an adversarial condition with the anti-sycophancy controller enabled (DS with Anti). For each condition, I computed three metrics: Accuracy, Sycophancy Rate, and FOG Rate. In the current implementation of the notebook, FOG Rate is defined as the fraction of cases where the final answer is both correct and sycophantic at the same time, rather than the

original trace-based definition in the assignment prompt. Concretely, the code counts a case toward FOG when both `is_correct` and `is_sycophantic` are `True`.

The final results are summarized below.

Condition	Accuracy	Sycophancy Rate	Printed FOG Rate
D0 Baseline (no hint, no Anti)	80.0%	40.0%	40.0%
DS Adversarial (no Anti)	100.0%	20.0%	20.0%
DS Adversarial (with Anti)	100.0%	40.0%	40.0%

Under this implementation, FOG Rate is always less than or equal to Sycophancy Rate by construction, because FOG is computed from the intersection of correctness and sycophancy. A case cannot be counted as FOG unless it has already been counted as sycophantic. In my results, however, FOG Rate is exactly equal to Sycophancy Rate in all three settings. This means that every answer flagged as sycophantic was simultaneously marked as correct by the current string-matching evaluation rule.

This pattern leads to an important interpretation. In the present notebook version, sycophancy does not necessarily correspond to a fully wrong final answer. Instead, it suggests that the answer may contain language or values that match the adversarial hint while still also containing the ground-truth value used by the heuristic checker. In other words, the model may be showing a softer form of sycophantic behavior: it partially accommodates the misleading hint in the final response, but the answer still remains "correct" according to the current matching criterion. This explains why the overlap between correctness and sycophancy is complete in the reported results.

Comparing the three conditions, the baseline system without adversarial hints achieves only 80% accuracy, showing that the unguarded agent is imperfect even in normal conditions. In the adversarial setting without the anti-sycophancy loop, the model attains 100% accuracy and a lower sycophancy rate of 20%. With the anti-sycophancy controller enabled, accuracy remains at 100%, but the measured sycophancy rate rises to 40%. Based on the current heuristic, this suggests that the controller did not reduce overlap between correct answers and hint-following language. Instead, it appears to have preserved answer correctness while still allowing adversarial content to be echoed in the final response often enough to trigger the sycophancy detector.

Overall, the evaluation shows that metric interpretation depends heavily on the exact implementation of FOG Rate. In the updated version of the notebook, FOG Rate measures the overlap between correctness and sycophancy, so equality between the two rates indicates that all sycophantic cases remain factually correct under the current answer-matching rule. This is different from a stricter trace-based interpretation, but it is consistent with the code and results reported here.

7. Conclusion

This homework implemented a small financial analysis agent for the synthetic NovaTech corpus by combining retrieval, explicit reasoning, a Judge module, and a feedback controller

designed to mitigate sycophancy. The system was intended to answer financial questions using retrieved evidence, perform structured reasoning with tools, and react to possible trace – answer mismatches through a multi-turn correction loop.

The empirical results show that the system can achieve strong answer-level accuracy across both normal and adversarial settings. In the final evaluation, the baseline condition reached 80% accuracy, while both adversarial conditions reached 100% accuracy. This indicates that the overall pipeline was able to recover the relevant financial facts and produce answers that matched the ground-truth values under the notebook's current evaluation heuristic.

At the same time, the sycophancy-related metrics show that answer correctness alone is not sufficient to guarantee robust behavior. The measured Sycophancy Rates were 40% in the baseline condition, 20% in the adversarial condition without the anti-sycophancy loop, and 40% in the adversarial condition with the loop enabled. Using the updated implementation of FOG Rate, the FOG values exactly matched the Sycophancy Rates in all three cases. This means that every answer identified as sycophantic was also classified as correct. Therefore, in this notebook version, sycophancy should be interpreted less as outright factual failure and more as an undesirable tendency to absorb or repeat misleading user cues while still preserving enough correct information to pass the answer checker.

This result is important because it highlights the difference between surface correctness and robust reasoning behavior. A system may still appear accurate according to a string-matching metric, yet its final answer may reveal susceptibility to user framing or adversarial hints. In other words, the agent can be simultaneously correct and behaviorally weak. The equality between FOG Rate and Sycophancy Rate in the current implementation makes this especially clear: the problem is not always that the model forgets the truth, but that it may present the truth in a way that is still contaminated by the hint.

In conclusion, this assignment demonstrates that building safe financial agents requires more than retrieval and tool use. It also requires evaluation methods that distinguish between factual correctness, reasoning integrity, and final-answer alignment. The current system successfully answers questions, but the remaining sycophancy signals suggest that further improvements are needed in the judge logic, feedback strategy, and especially in the metric design used to detect subtle failures. Even when the final numeric answer is correct, the model may still display behavior that is unreliable in high-stakes financial settings. That observation is the main lesson of this project.